

TYPATCH: Transforming Patches into Typestate Rules for Kernel Bug Detection

Ruoyu Wang^{*}
College of AI
Tsinghua University
Beijing, China
The University of Hong Kong
Hong Kong, China
ruoyu_wang@connect.hku.hk

Tuo Li
Tsinghua University
Beijing, China
islituo@163.com

Jia Li[†]
College of AI
Tsinghua University
Beijing, China
jia_li@mail.tsinghua.edu.cn

Abstract

Historical Linux kernel patches capture defect knowledge that applies beyond their original repair sites. Recent work has shown that large language models (LLMs) can generate static-analysis checkers from historical patches and use them to uncover new kernel bugs. However, complete-checker generation requires the model both to recover the defect semantics expressed by a patch and to implement sophisticated program-analysis machinery, including object tracking, alias analysis, path-state maintenance, and interprocedural propagation. Coupling these responsibilities in a single end-to-end code-generation task can turn a simple defect rule into an unstable and expensive analyzer-implementation problem.

To address this problem, we present TYPATCH, which decouples patch-specific defect semantics from analyzer implementation. An LLM translates each patch into a typestate rule specifying its tracked object, actions, guards, transitions, and violations. A shared backend then executes these rules, binding their actions to program events, tracking object identity across aliases, propagating typestate along program paths, and producing reports for all rules. On Linux v6.16, TYPATCH finds 559 distinct bugs, 121 of which have been confirmed by kernel developers. In a matched 38-patch comparison with the state-of-the-art complete-checker construction workflow, TYPATCH uses 88.3–90.1% fewer generation tokens, while its initial report pools achieve 3.42–14.95× the precision of those produced by that workflow.

CCS Concepts: • Security and privacy → Systems security; • Software and its engineering → Automated static analysis.

Keywords: Static Analysis, Large Language Models, Linux Kernel, Bug Detection

1 Introduction

The Linux kernel imposes many subsystem-specific rules on how objects are created, checked, used, and released. Violating these rules can cause crashes, resource leaks, and

memory-safety bugs. The kernel is large and highly configurable, and many code paths are exercised only with specific hardware or under rare error conditions. Consequently, ordinary test workloads cover only a fraction of the relevant behavior, making bugs in the remaining paths difficult to uncover through code review and dynamic testing alone.

Static-analysis tools such as Smatch, Coccinelle, and the Clang Static Analyzer can continuously search large portions of the kernel for defects [7, 31, 35]. However, the classes of bugs they detect largely depend on the rules that experts have already implemented. For a new long-tail defect pattern, developers must not only understand the relevant kernel semantics, but also encode those semantics as an executable program analysis. As a result, much of the defect knowledge already present in historical fixes has not been converted into persistent detection capability for the rest of the kernel.

Historical fixes provide a distinctive source of knowledge for constructing new static-analysis rules. A patch records the developer’s explanation of the root cause, the concrete repair action, and the behavioral difference between the buggy and fixed versions. The knowledge expressed by a patch is often not confined to the modified lines; it can capture a transferable relationship among an object, the operations performed on it, and the conditions under which those operations are valid. Automatically extracting this relationship would allow a single repair to guide automated detection of similar bugs across the kernel.

KNIGHTER represents the state of the art in automated patch-to-checker generation. It uses an LLM to understand Linux repair patches, generate detection plans, and implement complete Clang Static Analyzer checkers. This design combines automatic bug-pattern learning with scalable repository-wide analysis: the generated checkers have discovered many previously unknown Linux kernel bugs [43].

Despite this success, complete-checker generation remains unreliable and expensive. KNIGHTER’s own failure analysis identifies checker implementation as its largest source of failure. Of 22 patch commits that yielded no valid checker, 13 (59%) were attributed to inaccurate implementation, compared with 2 to inaccurate bug-pattern analysis and 7 to inaccurate plans [43]. In our matched 38-patch study, KNIGHTER

^{*}Work done during an internship at Tsinghua University.

[†]Corresponding author.

produces only 27 valid checkers out of 167 candidates with GPT-5.5 and 23 out of 198 with DeepSeek-v4-pro. Failed candidates trigger additional generation, compilation repair, and validation. The two runs consume 7.28 and 9.88 million generation tokens. A low yield of valid artifacts leaves much of the recovered defect knowledge unused, while implementing a separate checker for every patch limits how much repair history can be turned into persistent detection capability.

This fragility stems from coupling patch understanding to analyzer implementation. The model first infers the patch-specific object-state relation and then implements it as a complete checker. Implementing it requires framework callbacks and state containers for alias-aware, path-sensitive interprocedural propagation and reporting. These analysis mechanisms are difficult but largely common across patches [24, 25]; an implementation error can therefore invalidate a checker after successful semantic recovery, and each retry regenerates similar analyzer machinery.

To remove this coupling, TyPATCH separates the two responsibilities. The LLM generates only the patch-specific defect semantics as a structured tpestate rule, while a shared backend supplies the program-analysis mechanisms for all accepted rules [38].

Figure 1 summarizes the two-stage pipeline. First, for each patch, TyPATCH combines the commit message, code diff, and related source context as evidence. An LLM then generates a tpestate rule specifying the tracked object, key actions and guards, state transitions, violation state, and report evidence. Deterministic validation and repair then produce a valid rule or NoRule. Second, the shared backend executes the accepted rule pool over the kernel’s LLVM IR and control-flow graph (CFG). It binds actions to program events, tracks objects and tpestate along paths, and emits bug reports.

Implementing TyPATCH poses two central challenges. First, the rule representation must capture different object-state relations and bind them precisely to program events. TyPATCH’s rule representation explicitly defines tracked objects, actions and guards, state transitions, and report evidence. Second, the backend must execute many heterogeneous rules consistently and efficiently. TyPATCH provides shared action matching, alias-aware tracking, path-sensitive propagation, report construction, and grouped execution.

We select 100 historical Linux fixes across eight bug families and use them as seeds for rule generation. The resulting rule pools find 559 distinct bugs in Linux v6.16, 121 of which have been confirmed by kernel developers. Given the same 100 seeds, GPT-5.5, DeepSeek-v4-pro, and Claude Opus 4.8 generate tpestate rules for 91, 74, and 84 patches, respectively. We compare TyPATCH with KNIGHTER on KNIGHTER’s public 38-patch dataset, using the same model families and Linux v6.16 target. Across GPT-5.5 and DeepSeek-v4-pro, KNIGHTER consumes 8.56× and 10.09× as many generation tokens, respectively. On the resulting initial report pools, TyPATCH achieves 4.67% and 9.11% precision, compared with

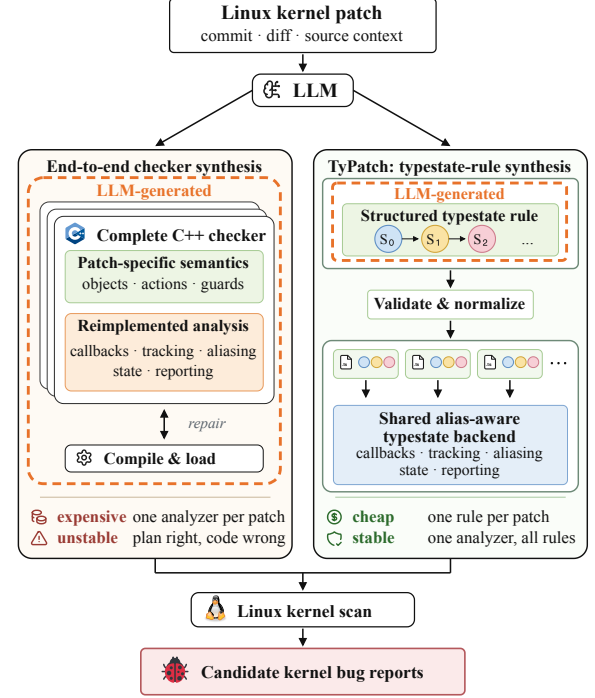


Figure 1. Generate the patch-specific rule; reuse the analyzer. Complete-checker generation asks the model to produce both defect semantics and checker implementation. TyPATCH instead generates a structured tpestate rule and executes it with a shared backend.

0.31% and 2.66% for KNIGHTER, under GPT-5.5 and DeepSeek-v4-pro, respectively. We further review all 249 generated rules: 236 preserve the core defect relation in their seed fixes, while the remaining mismatches concentrate in object identity, failure predicates, action binding, and state topology.

We make the following three contributions:

- **Rule representation.** We introduce a tpestate representation for patch-derived defect semantics with explicit program bindings, separating them from general program-analysis mechanisms.
- **System.** We design and implement TyPATCH, which constructs and checks rules from commit messages, code diffs, and source context and executes them with a shared alias-aware, path-sensitive backend.
- **Evaluation and impact.** We evaluate three LLMs on 100 Linux fixes, compare TyPATCH with KNIGHTER on 38 matched patches, and analyze rule fidelity; the resulting rules find 559 distinct bugs in Linux v6.16, 121 of which have been confirmed by kernel developers.

2 Motivation

Patch-to-checker generation combines two distinct tasks: recovering the defect relation expressed by a patch and implementing that relation as a correct analyzer. The former

requires semantic understanding of the commit evidence; the latter requires framework-specific callbacks, object identity, alias handling, and path-state updates. We use one Linux repair to expose the gap between them.

2.1 Correct Plan, Incorrect Checker

Figure 2(a) shows a null-pointer dereference in the cgbc hwmon driver and the check later added upstream. The function stores the return value of `devm_kzalloc()` in `hwmon->sensors`, copies that field into the local cursor `sensor`, and dereferences the cursor while initializing the array. Because the allocator may return `NULL`, the added guard returns before the local assignment on the failure path. The continuing path therefore establishes that the allocation reached through `sensor` is non-null.

The KNIGHTER plan in Figure 2(b) recovers this relationship: the allocator return flows through the structure field and local alias to the unsafe use. Implementing that plan as a CSA checker is a different task. The checker must preserve one object identity across these representations, attach the non-null fact only to the continuing branch, and recover the same object state at the later field access through the appropriate CSA callbacks and ProgramState keys.

This distinction appears consistently in the generated artifacts for this patch. All 20 generated plans identify the missing null check before the subsequent dereference, and all 20 generated checkers compile. Yet only 2 of the 10 checkers generated with GPT-5.5 and none of the 10 generated with Opus 4.8 distinguish the buggy seed from its fixed version. For this patch, 18 of 20 candidates therefore fail after recovering the missing-check relationship, when that relationship must be implemented through CSA memory regions, callbacks, and path states.

The generated checker in Figure 2(c) stores the unchecked state using a region derived from the destination expression, but later queries the alias map using a different region derived from the bound value. These expressions resolve to different CSA regions, so the alias update for `sensor` is skipped and the checker reports neither the buggy nor the fixed seed. This is not a failure to recover the patch semantics: the plan already contains the correct allocation, alias chain, guard, and use. It arises because complete-checker generation requires each generated checker to rebuild CSA-specific object and path machinery. Every checker must independently select callbacks, region keys, state containers, and transitions; compilation repair cannot ensure that these choices implement the recovered plan. Coupling patch understanding to analyzer implementation therefore turns a small defect relation into an unstable analyzer-implementation task.

2.2 Generate the Rule, Reuse the Analyzer

This coupling can be avoided by changing the generation boundary. Across patches, the tracked object, actions, guards,

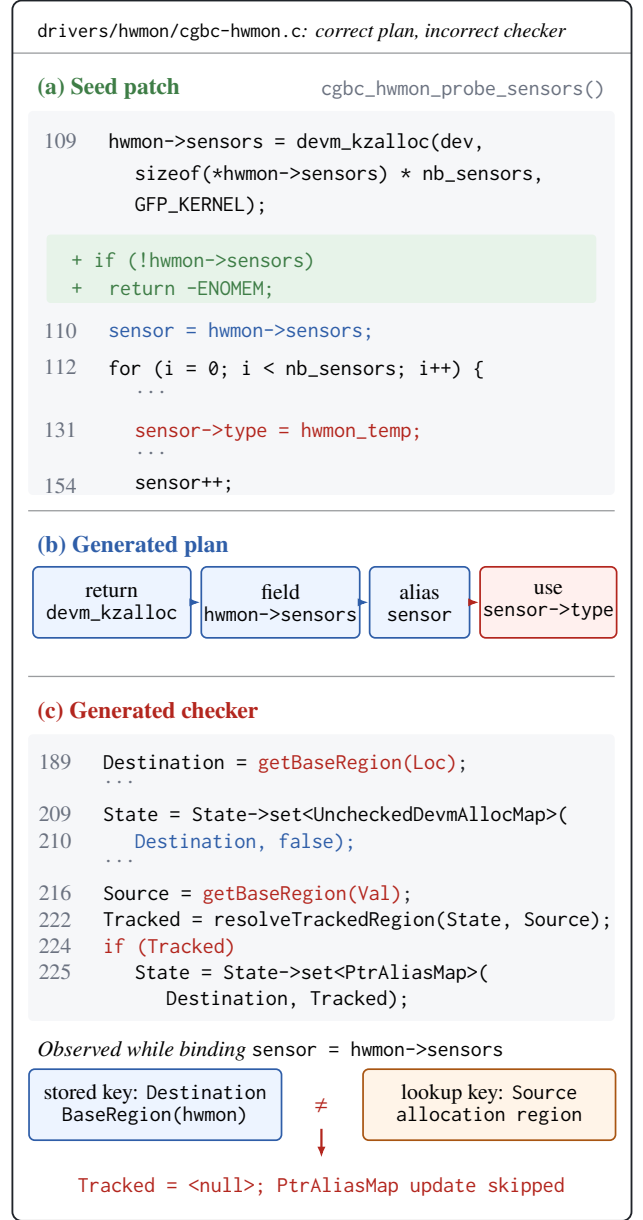
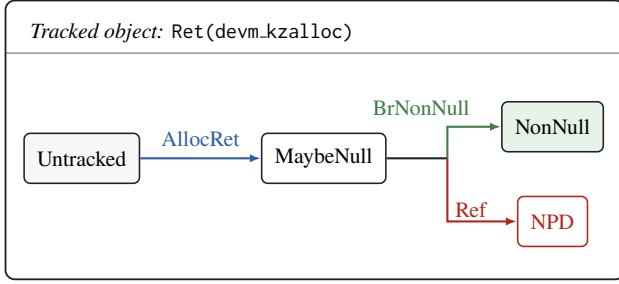
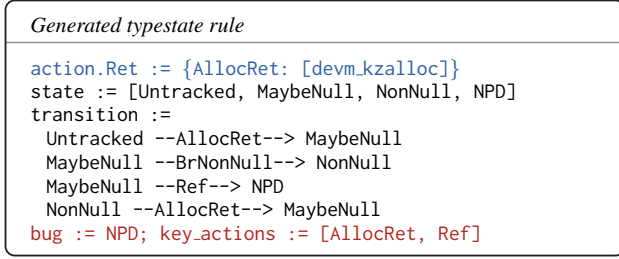


Figure 2. Correct plan, incorrect checker for the cgbc patch. (a) The fix checks the allocation before aliasing and dereference. (b) The plan preserves the allocator-to-use chain. (c) The checker stores state under a `Loc`-derived key but queries it with a `Val`-derived key, so the lookup misses and the alias-map update is skipped.

and transitions vary, whereas object propagation, alias handling, and path-state maintenance are common analysis mechanisms. TYPATCH therefore asks the model to emit only a structured typestate rule and implements those common mechanisms once in a shared backend.



(a) Patch-derived state topology.



(b) Generated rule.

Figure 3. The cgbc typestate rule and its bindings to `devm_kzalloc()`, the non-null guard, and the dereference.

For the same cgbc patch, the generated rule binds the source to `devm_kzalloc()`, the guard to its non-null successor, and the violation to the later dereference. The backend carries the object through the structure field and local alias and applies the non-null transition only to the continuing branch—the mechanics that the generated checker failed to encode. GPT-5.5 and Opus 4.8 each generated this rule in one attempt; Figure 3 shows its state topology and action bindings for this patch.

This example makes this separation concrete. The next section formalizes it through TyPATCH’s rule representation, construction process, and shared typestate analysis.

3 Method

Overview. TyPATCH generates patch-specific typestate rules and executes them with a shared backend. Each rule records the tracked object, state-changing actions and guards, state transitions, and report evidence. The backend binds rule actions to program events and tracks objects across aliases. It propagates typestate and constructs reports.

Figure 4 shows how these components interact. For each patch, TyPATCH combines the commit message, code diff, and source context. It returns either a validated rule or NoRule. The backend loads all accepted rules and analyzes the kernel’s LLVM IR and CFG. The rule representation connects patch-derived defect knowledge to the shared backend. It

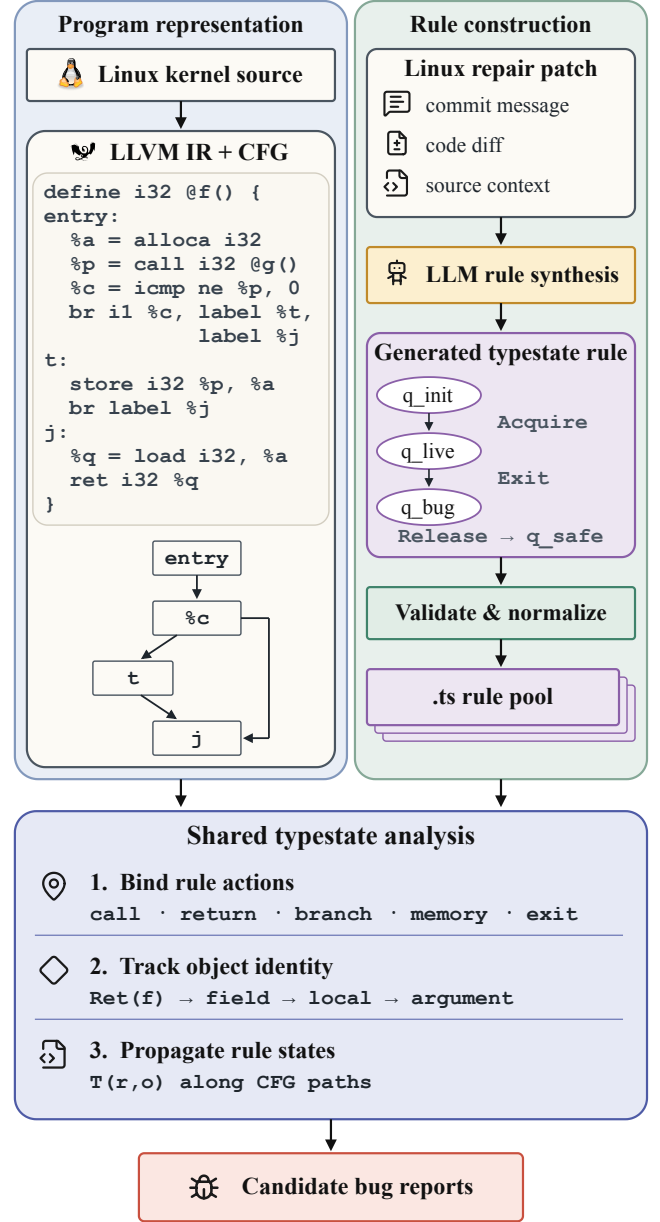


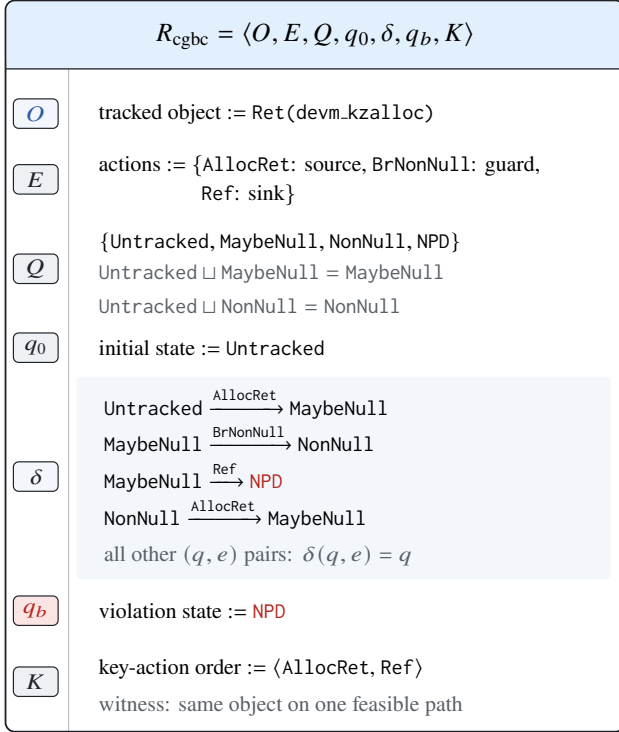
Figure 4. Architecture of TyPATCH.

expresses that knowledge in a form the backend can execute. We first define this representation, then describe its construction and execution.

3.1 Typestate Rule Representation

The shared backend requires a precise description of the defect relation recovered from a patch. The rule representation provides this description as a state machine with the bindings needed to execute it. The complete rule is

$$R = \langle O, E, Q, q_0, \delta, q_b, K \rangle.$$

Figure 5. The cgbc rule instantiated in R .

Its typestate machine is (E, Q, q_0, δ, q_b) . Here, E is the action alphabet, Q is a finite abstract state domain, and $q_0 \in Q$ is the initial state. The function $\delta : Q \times E \rightarrow Q$ defines state transitions, and $q_b \in Q$ is the violation state.

The remaining components connect the machine to a program and to a report. O specifies how a starting action selects the program value that represents the tracked object. The evidence contract is $K = (\bar{e}, \Phi)$. Here, $\bar{e} = \langle e_1, \dots, e_m \rangle$ is the ordered key-action sequence, and Φ contains report constraints such as path consistency or distinct action locations. The backend drops a candidate only if it proves a constraint impossible. Thus, δ describes how matched actions affect state, whereas K describes the evidence required for a report.

The domain Q also has a deterministic join $J_Q : Q \times Q \rightarrow Q$ for CFG merges. For a rule r with state domain Q_r , we write $J_r = J_{Q_r}$ for its join operator. A generated rule may declare merge cases, and normalization completes them with a shared fallback policy. Explicit cases take precedence, and the violation state is absorbing. For scalar uninitialized-use rules, joining an initial and non-initial state yields the initial state; for lifecycle rules, the same pair preserves the non-initial obligation. Any remaining unmatched pair follows the CFG’s deterministic predecessor order. This makes join behavior an explicit part of the normalized state domain.

Figure 5 gives an instance of R for the cgbc patch in Figure 2(a). In this rule, O is the return value of `devm_kzalloc()`, and E contains the allocation return, non-null branch, and

later dereference. The transition function places the allocation in `MaybeNull`. The continuing arm of the guard changes the state to `NonNull`, while an unchecked dereference reaches `NPD`. A later `AllocRet` resets `NonNull` to `MaybeNull` for the new allocation result. In the figure, $\sqcup$ denotes the join operator J_Q . The two displayed equations are the rule-specific merge cases; all other state pairs use the shared fallback policy described above. The evidence contract K requires the allocation and dereference to refer to the same object along a path not proven infeasible.

The bindings attached to O and E make the state machine executable. An object may enter the analysis as a function return, an argument or out parameter, a field value, or a managed allocation. A call action names the callee and selects the argument and optional field that carries the object. A return action selects the call result, and a branch action selects an outgoing CFG edge. A memory action selects a load, store, or dereference. An exit action selects a function return. For example, `Call[of_node_put, arg0]` denotes a release of the object passed as the first argument to `of_node_put()`; “release” alone would not identify a program event.

The rule omits unbound operations that only transport an object between state-changing actions. The shared analysis handles unbound loads, stores, field address computations, casts, and actual-to-formal transfers. A memory operation appears in E only when the repair assigns it typestate significance. A rule specifies its state joins and witness constraints, while the shared analyzer implements the alias propagation and feasible-path confirmation needed to enforce them.

3.2 Constructing Rules from Patches

Constructing a rule requires evidence that is rarely stated in one place. The commit message states the defect and repair intent. The diff identifies the operation that changes behavior. The surrounding pre-fix source reveals the object flow and conditions on which the repair depends. `Typatch` combines these sources before generation. When a macro, wrapper, or cleanup declaration hides the state-changing operation, context construction resolves its definition. Each excerpt records its file and line range, which source grounding later uses to verify APIs named by the generated rule.

The LLM receives this evidence together with the typed rule schema. It generates one complete candidate rule. The candidate names the tracked-object origin, declares its actions and guards, and supplies the state machine and violation evidence in a single structured artifact. It may instead return `NoRule` when the repair depends on semantics outside the rule interface, such as numeric bounds.

The validation pipeline has two stages separated by normalization. Before normalization, *schema validation* checks field types, required action fields, and references to declared states and actions. *Source grounding* then checks every function named by a call or return action against the collected

evidence. Once both checks pass, normalization canonicalizes the candidate and fills unlisted state-action pairs with identity transitions.

After normalization, *state-machine validation* checks that q_b is reachable and that replaying $\bar{e}$ from q_0 reaches q_b . *Backend validation* then requires a source action that identifies the tracked object, rejects unsupported action combinations, and verifies that the rule can be translated into the backend representation. Across the two stages, argument positions, field paths, and branch or exit kinds are also checked structurally. The first stage establishes that a candidate is well formed and grounded; the second establishes that the normalized rule is internally consistent and executable.

Validation failures are reported at the rule-field level. A revision request contains the original patch evidence and the rejected candidate. It also reports diagnostics such as an ungrounded callee, an unreachable violation state, or a key-action sequence that does not form a transition path. The model may revise the candidate twice, and every revision passes through the complete validation pipeline again. If no candidate succeeds, construction returns `NORULE`.

Once the normalized rule passes state-machine and backend validation, TyPATCH serializes it in the declarative `.ts` format consumed by the shared analyzer. Normalization preserves the tracked object, bound actions, state-changing transitions, violation state, and evidence contract. Appendix A.1 gives the complete construction algorithm, context limits, example selection, normalization, and validation details.

3.3 Shared Typestate Analysis

A generated rule specifies the program events that change typestate. Executing it over LLVM IR requires the backend to (1) bind each action to a concrete event and selected value, (2) preserve object-specific state across aliases and control flow while discarding histories proven inconsistent with any path, and (3) execute heterogeneous rules without mixing their states or evidence. TyPATCH uses shared action matching and alias-aware tracking to connect rule actions to program events and objects. Path-sensitive propagation carries rule states along the CFG, and evidence screening removes candidates proven infeasible. Grouped execution applies this machinery across multiple rules. Algorithm 1 gives the complete workflow. It joins incoming facts, transfers object identity, applies matched actions, and screens the resulting candidates. We next explain the three mechanisms that support this workflow.

Binding actions to program events. A rule describes state-changing events, whereas the backend operates on LLVM instructions and CFG edges. The loader indexes actions by binding form and resolves each match to an instruction or edge, a selected value, and an optional field path. Return actions select call results; call actions select arguments or out-parameter slots; and memory actions select

Algorithm 1: Shared Typestate Analysis

Input: LLVM link unit U , rule set $\mathcal{R}$

Output: Ordered source-level reports D

```

1 # Prepare the analysis unit
2  $G \leftarrow \text{BuildAnalysisCFG}(U, \mathcal{R})$  by removing loop and
   return backedges
3  $W \leftarrow []$ ;  $D \leftarrow []$ 
4 initialize entry-edge states  $\Gamma = \langle A, T, H, B \rangle$ 
5 foreach node  $n$  in topological order of  $G$  do
6   # Join predecessor facts
7    $A_n \leftarrow \text{JoinAliasGraphs}(\{A_p : p \in \text{pred}(n)\})$ 
8    $(T_n, H_n, B_n) \leftarrow \text{JoinRuleFacts}(\{\Gamma_p : p \in$ 
      $\text{pred}(n)\}, A_n, \{J_r\})$ 
9   # Transfer object identity
10   $A_n \leftarrow \text{AliasTransfer}(n, A_n)$ 
11   $\Gamma_n \leftarrow \langle A_n, T_n, H_n, B_n \rangle$ 
12  # Apply node and exit actions
13  foreach  $(r, e, v, f) \in \text{MatchActions}(n, \mathcal{R})$  do
14     $\text{Advance}(\Gamma_n, r, e, v, f, n, W)$ 
15  # Fork successor states and apply edge
     actions
16  foreach outgoing edge  $\ell$  of  $n$  do
17     $\Gamma_\ell \leftarrow \text{copy}(\Gamma_n)$ 
18    foreach  $(r, e, v, f) \in \text{MatchActions}(\ell, \mathcal{R})$  do
19       $\text{Advance}(\Gamma_\ell, r, e, v, f, \ell, W)$ 
20    store  $\Gamma_\ell$  on  $\ell$ 
21 # Discard paths proven inconsistent
22 foreach candidate  $d = (r, o, H) \in W$  do
23   if  $\Phi_r = \emptyset$  or  $\neg \text{RefutePath}(G, d, K_r)$  then
24     append  $d$  to  $D$ 
25 return  $D$ 

```

loads, stores, or dereferences. Guard actions bind to outgoing edges, and exit actions are projected over active tracked objects. All rules share this matcher while supplying different bindings and transition tables.

$\text{MatchActions}(\ell, \mathcal{R})$ returns (r, e, v, f) for actions bound to LLVM instructions or outgoing CFG edges.

Preserving object identity. Matched actions belong to one rule execution only when their selected values represent the same object. The *AliasGraph* $A = \langle N, F, \rho \rangle$ makes this identity explicit: N contains abstract objects, F contains labeled pointee and field edges, and $\rho : V_{\text{IR}} \rightarrow N$ maps LLVM values to objects. Resolving (v, f) obtains $\rho(v)$ and follows field path f .

Stores replace a destination's ref edge, loads follow it, and `getelementptr` creates or follows field edges. Casts preserve

nodes; ϕ and select do so only when all incoming values resolve to one node. At resolved calls, actuals and formals share nodes, allowing stores through out parameters to remain visible after return. Summaries similarly connect recognized driver-data setter/getter pairs.

At CFG joins, predecessor nodes containing the same LLVM value are unified, followed recursively by destinations reached through the same field label. Rule slots are remapped to the unified nodes before their states are joined. State therefore survives field, local-alias, and interprocedural transfers without encoding those transfers in each rule.

Preserving path-specific state. Object identity alone is insufficient when a guard or discharge applies on only one branch. TyPATCH removes loop and return backedges and traverses the resulting acyclic interprocedural graph in topological order. Each analysis edge carries $\Gamma = \langle A, T, H, B \rangle$: the AliasGraph A , rule-indexed object states T , supporting histories H , and branch facts B . One object can thus carry independent state slots for several rules.

For rule r , δ_r , q_b^r , J_r , and $K_r = (\bar{e}_r, \Phi_r)$ denote its transition function, violation state, join, and evidence contract. On a matched action, $\text{Advance}(\Gamma, r, e, v, f, \ell, W)$ resolves the selected object, appends the occurrence to its history, and applies δ_r . A starting action first initializes the object’s rule slot at q_0^r ; a projected exit advances every active object. Reaching q_b^r adds a candidate to W . Exact transitions take precedence over normalized identity self-loops.

Node actions update state before propagation. The analyzer copies the state to each successor and applies guard actions only to their bound edges. At CFG joins, it merges incoming tpestates using J_r . With conditional-state tracking enabled, it can retain separate states for complementary null and non-null branches on the same value until a later guard selects one or J_r merges them. Each retained state carries one representative history.

Since merged facts may originate from different paths, reaching q_b^r produces a candidate report. When Φ_r requests screening, the backend replays the key-action sequence $\bar{e}_r$ along individual paths in the relevant CFG slice. Each path maintains its own alias graph, tpestate, and branch facts during replay. This checks whether the required actions can operate on the same object in order along one path, rather than relying on the merged scan state alone. The backend rejects a candidate only if it proves that no path satisfies the constraints in K_r , including field paths and distinct action locations. Unknown cases are retained. Reports record the rule, object, bound actions, history, and source sites.

The same traversal executes compatible rules together in one link unit. They share one AliasGraph, while every object carries independent rule-indexed states and histories. Action indices dispatch each event to all matching rules after paying its object-transfer cost once. Adding a rule therefore does not

require another checker implementation or another model call during the scan.

We now trace the cgbc example in Figures 2(a) and 3 through Algorithm 1. The allocation action initializes the return from `devm_kzalloc()` in *MaybeNull*. Alias transfer then carries the same object through `hwmon->sensors` to `sensor`. In the fixed seed, the non-null edge changes its state to *Non-Null*, while the null edge returns before the dereference. In the pre-patch seed, no guard performs this transition. The later dereference therefore changes *MaybeNull* to *NPD* and creates a candidate. Evidence screening retains it because the allocation, alias transfers, and dereference form a feasible same-object path.

4 Evaluation

We explore the following research questions for TyPATCH:

RQ1. Can TyPATCH find real-world Linux kernel bugs?

RQ2. How do rule and checker construction compare in artifact yield, generation cost, and initial report quality?

RQ3. How faithfully do rules preserve seed semantics?

4.1 Experimental Setup

Datasets and models. All scans target Linux v6.16 at commit 98a11dadac64. RQ1 and RQ3 use 100 historical Linux fixes across eight bug families, with rules generated independently by GPT-5.5, DeepSeek-v4-pro, and Claude Opus 4.8. RQ2 compares both systems with GPT-5.5 and DeepSeek-v4-pro on the 38 commits in KNIGHTER’s public benchmark spanning six shared bug families.

Environment. We performed all kernel-wide scans on a dual-socket server with two 12-core Intel Xeon Silver 4310 CPUs, 256 GB RAM, and Ubuntu 24.04, using eight workers.

4.2 RQ1: Linux Bug Finding

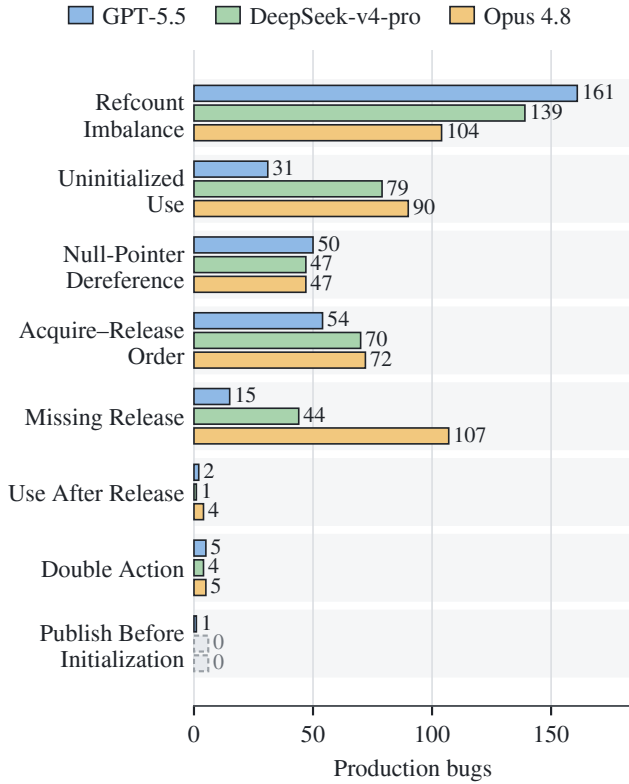
Across the three RQ1 rule pools, TyPATCH finds 559 distinct bugs in Linux v6.16, 121 of which have been confirmed by kernel developers. Before cross-model deduplication, GPT-5.5, DeepSeek-v4-pro, and Opus 4.8 contribute 319, 384, and 429 TP components, respectively, for a total of 1,132. The findings span all eight evaluated tpestate families across networking, media, sound, GPU, PHY, and IIO.

Given the same 100 seed fixes, GPT, DeepSeek, and Opus generate rules for 91, 74, and 84 patches, respectively. Table 1 breaks this coverage down by family. Missing-release, refcount-imbalance, and acquire-release-order seeds translate consistently across models. The largest coverage differences occur for uninitialized-use, use-after-release, and publish-before-initialization seeds.

Figure 6 shows the distribution of these model-specific TP components. Refcount imbalances are the largest group for GPT and DeepSeek, while Opus contributes substantially

Table 1. Generated tpestate rules for RQ1’s 100 patches.

Bug family	Input	GPT	DS	Opus
Null-Pointer Dereference	20	18	17	19
Double Action	20	19	12	15
Missing Release	17	17	15	17
RefCount Imbalance	16	16	15	16
Uninitialized Use	10	8	8	4
Use After Release	10	7	3	7
Acquire–Release Order	4	4	4	4
Publish Before Initialization	3	2	0	2
Total	100	91	74	84

**Figure 6.** RQ1 true-positive components by bug family; gray stubs denote zeros.

more missing-release findings. In our taxonomy, *RefCount Imbalance* denotes unmatched get/put-style references; *Missing Release* denotes an unreleased acquired resource.

Of the 559 distinct bugs, 196 are found by all three model-derived rule pools, 181 by exactly two, and 182 by one. Thus, 377 of 559 bugs (67.4%) are surfaced by rule pools from at least two models, while each model also contributes findings absent from the other two. Table 2 reports the corresponding model-specific component pools and their source-adjudicated precision.

Table 2. Model-specific RQ1 report pools after deduplication. Unique TPs are not found by the other two models.

Model	Reports	TP	Unique TP	FP	Precision
GPT-5.5	3,662	319	54	3,343	8.71%
DeepSeek-v4-pro	4,842	384	40	4,458	7.93%
Opus 4.8	5,655	429	88	5,226	7.59%

Constructing each model-specific rule pool consumes 2.52–3.12 million model tokens, including repair attempts; Appendix A.4 provides the input/output breakdown.

Case studies. Figure 7 presents two representative transfers from historical seed repairs to developer-confirmed findings. The first carries a nullable-object check from RDMA to WWAN; the second carries an all-path initialization requirement from ALSA to a media I2C helper.

Cross-subsystem nullable allocation. The seed in Figure 7(a) checks the return from `skb_clone()` before an RDMA receive path converts it into packet state. The resulting nullable-object rule exposes the same unchecked return in the WWAN control transmit path in panel (b), where allocation failure reaches `cloned->len` under different surrounding control flow. The accepted fix adds the missing guard while preserving the driver’s partial-write return convention.

Cross-stack output initialization. The ALSA seed in Figure 7(c) initializes a returned status on every path. The resulting rule finds a media-driver variant in panel (d): `solo_i2c_readbyte()` ignores the number of messages completed by `i2c_transfer()` and returns data even when the transfer is short. This can expose an uninitialized stack byte to chip detection, V4L2 input-status queries, and ALSA gain controls.

Answer to RQ1. TyPATCH finds 559 distinct bugs in Linux v6.16, including 121 confirmed by kernel developers. These bugs span all eight evaluated tpestate families and multiple kernel subsystems; 377 (67.4%) are found by rules generated by at least two models.

4.3 RQ2: Rule vs. Checker Construction

We run TyPATCH as described in Section 3.2 and KNIGHTER with its published default configuration. *Candidates* count generated rule or checker versions, including repairs; *Artifact yield* counts input patches whose workflow produces a rule or checker accepted by its construction pipeline and included in the kernel scan. Generation-token cost includes all construction attempts. We execute every resulting artifact and report the TP, FP, and precision of the *initial report pools* before optional report-level post-processing.

Artifact construction. Table 3(a) shows the same result under both models: TyPATCH completes construction for more patches while making fewer model calls and consuming

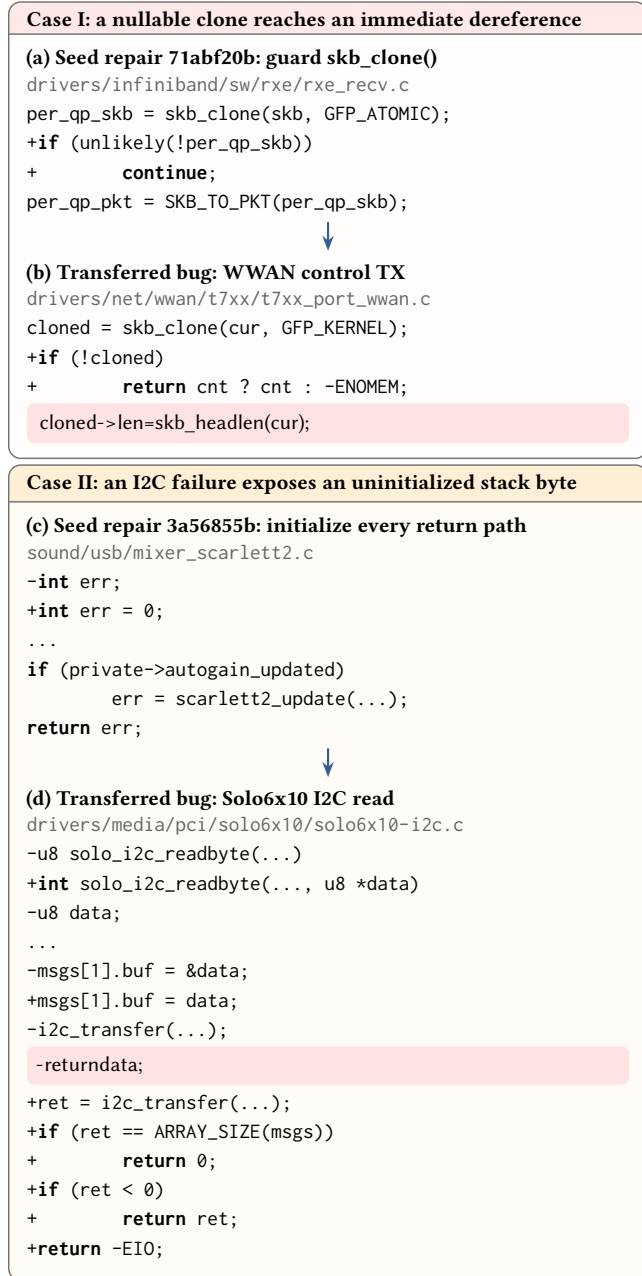


Figure 7. Two seed-to-bug transfers. Panels (a,b) transfer a nullable-object rule from RDMA receive to WWAN transmit; panels (c,d) transfer an all-path initialization rule from ALSA control logic to a media-driver I2C helper. In each pair, the right panel shows the detected bug and its corresponding fix; red bands mark the unsafe sinks.

fewer tokens. Under GPT-5.5, it generates 57 rule candidates and constructs rules for 31 of 38 patches with 88 calls and 850.8K tokens; KNIGHTER generates 167 checker candidates and obtains 27 valid checkers with 596 calls and 7.28M tokens. Under DeepSeek-v4-pro, TyPATCH generates 62 candidates

Table 3. Matched-38 artifact construction and initial report quality. Token counts are in thousands (K).

(a) Artifact construction cost.						
Model	System	Can./Art.	Calls	Input	Output	Total
GPT-5.5	TyPATCH	57/31	88	769.6	81.2	850.8
	KNIGHTER	167/27	596	5,958.0	1,325.9	7,284.0
DeepSeek	TyPATCH	62/25	90	660.5	318.5	979.0
	KNIGHTER	198/23	826	5,610.1	4,271.6	9,881.7

(b) Initial report quality.				
Model	System	Reports	TP	Precision
GPT-5.5	TyPATCH	4,584	214	4.67%
	KNIGHTER	4,804	15	0.31%
DeepSeek	TyPATCH	1,735	158	9.11%
	KNIGHTER	2,178	58	2.66%

and constructs rules for 25 patches with 90 calls and 979.0K tokens, whereas KNIGHTER generates 198 candidates and obtains 23 valid checkers with 826 calls and 9.88M tokens. Thus, KNIGHTER makes 6.8–9.2× as many model calls, while the implemented TyPATCH workflow uses 88.3–90.1% fewer generation tokens.

Across both models, TyPATCH completes construction for more patches than KNIGHTER at a much lower average generation cost per artifact. Under GPT-5.5, the average costs are 27.4K tokens for TyPATCH and 269.8K for KNIGHTER; under DeepSeek-v4-pro, they are 39.2K and 429.6K, respectively.

Non-artifacts are concentrated in patches that require reasoning beyond a sequential object-state relation. Under GPT-5.5, six of seven non-artifacts are heterogeneous misuse patches: three require size or bounds reasoning, two encode a constant or API change without a temporal object relation, and one fails schema validation. The remaining UAF candidate omits the use action needed to form a violation. DeepSeek shows a similar pattern for value-, bounds-, and nonprotocol misuse seeds.

Initial report quality. Table 3(b) reports the quality of the initial report pools. Under GPT-5.5, TyPATCH produces 214 TP instances among 4,584 reports (4.67%), compared with 15 among 4,804 reports (0.31%) for KNIGHTER. Under DeepSeek-v4-pro, the corresponding results are 158 of 1,735 reports (9.11%) and 58 of 2,178 reports (2.66%). The lower precision under GPT-5.5 is concentrated in three broad rules: a sequential approximation of a concurrent UAF seed produces 1,753 reports, while two uninitialized-data rules produce 545 and 431. Together, these rules account for 2,729 (95.8%) of the 2,849 additional reports produced with GPT-5.5 relative to DeepSeek-v4-pro. Across the two models, TyPATCH therefore produces 2.72–14.27× as many TP instances at 3.42–14.95× the precision.

Answer to RQ2. Across the matched workflows, TyPATCH produces more artifacts under both models and uses 88.3–90.1% fewer generation tokens. Before report-level post-processing, it produces 2.72–14.27× as many true-positive report instances and achieves 3.42–14.95× the precision of the corresponding KNIGHTER results.

4.4 RQ3: Rule Fidelity and Misalignment

We define a rule as aligned with its seed patch when it preserves the tracked object and the source, discharge, and sink roles that constitute the defect relation. Generalizations of predicates or API bindings are also considered aligned if they preserve this core relation. We apply this definition to all 249 generated rules by comparing each rule with its seed commit message, code diff, and relevant source context.

Table 4(a) shows that 236 of 249 generated rules preserve their seed relation: 88 of 91 for GPT-5.5, 70 of 74 for DeepSeek-v4-pro, and 78 of 84 for Opus 4.8. All three models preserve the seed relation in more than nine out of ten generated rules, indicating that current LLMs can reliably translate patch evidence into TyPATCH’s structured rule representation.

Among the 236 aligned rules, we further distinguish exact reproduction from useful generalization. Of these rules, 159 preserve every audited object, action, predicate, scope, and topology field, while 77 broaden at least one predicate or API binding without changing the tracked object or the source-to-sink obligation. We count both as faithful because both preserve the defining object and action roles.

Table 4(b) summarizes the primary causes of the 13 mismatches. Action selection accounts for six: the rule chooses the wrong source, discharge, or sink operation. Four others track the wrong object or lifetime, often by selecting a neighboring owner or field. The remaining three change a defining failure predicate or the transition topology. Object and action selection therefore account for 10 of 13 mismatches, matching the generation challenges made explicit by *O* and *E* rather than failures in the shared alias or path implementation.

We further examine how semantic alignment relates to report quality. We group the reviewed, deduplicated findings according to the rules that reported them. We place a finding in the aligned group if every rule that reported it is aligned; otherwise, we place it in the misaligned group. One finding was reported by both aligned and misaligned rules, and we count it as misaligned. The 13 misaligned rules produced 340 deduplicated findings but only one TP, while aligned rules accounted for 1,131 of the 1,132 model-specific TPs. Thus, nearly all TPs came exclusively from seed-aligned rules.

Answer to RQ3. Across the three models, 236 of 249 generated rules (94.8%) preserve the core tracked-object relation and defining source, discharge, and sink roles of their seed fixes. The 13 mismatches are localized

Table 4. Seed fidelity and misalignment causes.

(a) Seed fidelity.				
Model	Rules	Aligned	Misaligned	Fidelity
GPT-5.5	91	88	3	96.7%
DeepSeek-v4-pro	74	70	4	94.6%
Opus 4.8	84	78	6	92.9%

(b) Primary misalignment causes.		
Cause	Differing rule decision	Rules
Object identity	tracked object or lifetime	4
Failure predicate	guard domain or branch polarity	1
Action binding	source, discharge, or sink operation	6
State topology	start, reset, or violation transition	2

to object identity, failure predicates, action bindings, and state topology. Aligned rules account for 1,131 of 1,132 model-specific TPs, indicating a strong association between seed fidelity and true-positive findings.

5 Discussion

A growing rule ecosystem. TyPATCH turns historical fixes into persistent, executable rules that can evolve with the kernel. Rules produced from different patches or models share the same interface and backend, so a new wrapper model, alias transfer, or path predicate can benefit the entire rule pool. Developer-confirmed findings and their fixes can become new seeds, allowing each analysis cycle to expand the available defect knowledge. Future work can further improve seed quality by combining patch series and follow-up fixes, and extend the rule representation to cross-interface protocols, numerical constraints, and concurrent histories.

Orthogonal report verification. RQ2 evaluates initial report pools before report-level postprocessing. LLM-based contextual analysis and triage can filter static-analysis alerts and refine overly broad checkers [26, 43], while candidate-level validation supports rule-guided bug discovery [41]. A downstream agent could inspect each report’s seed, bound actions, object history, and source context to reject infeasible or unrelated candidates without changing generation or execution.

Artifact Availability. To support artifact evaluation and reproducibility, we provide a project repository containing the rule synthesizer, shared backend, 100 historical repair commits, three frozen rule pools with their scope maps, and report deduplication code. The accompanying Docker image includes the Linux v6.16 source tree, LLVM IR, and compilation database needed to reproduce the scans. The artifact is available at <https://github.com/THU-Agent/TyPatch>.

6 Related Work

6.1 Traditional Static Analysis

The Linux kernel has long used static analysis to assist code review and defect detection. Coccinelle uses the Semantic Patch Language to describe patterns over syntax and control flow; Smatch provides an extensible rule interface for C and the Linux kernel; and the Clang Static Analyzer uses path-sensitive, interprocedural symbolic execution to let checkers observe program callbacks, maintain abstract states, and generate reports [7, 21, 31, 35]. These tools provide infrastructure for applying existing rules at scale, but new defect knowledge must still be encoded as corresponding rules.

Earlier general and kernel-specific analyzers established system-rule checking, path-sensitive verification, and scalable bug finding [3, 4, 6, 8, 10, 11, 13, 16, 32, 33, 39, 42].

Object-lifecycle constraints commonly require tpestate, alias, and path analysis. Tpestate associates an abstract state with a program object and updates that state in response to relevant actions; an operation that is not permitted in the current state constitutes a violation [5, 12, 15, 19, 20, 38]. PATA develops path-sensitive and alias-aware tpestate analysis for operating-system code, while SPATA improves the scalability of object and state propagation through interprocedural alias summaries [24, 25]. These systems show that accurately executing state rules requires specialized handling of object identity, path feasibility, and interprocedural propagation.

Another line of work automatically recovers rules from common behaviors, execution traces, revision histories, and source-level API usages [1, 14, 22, 27, 30, 34, 40, 44, 45]. Spinfer instead infers semantic patches from multiple code-change examples [36]. These approaches reduce manual rule construction but typically require many similar examples or restrict rules to predefined code patterns.

Historical bug fixes provide a more direct source of knowledge. Known vulnerable code and human fixes have been used to find unpatched clones and learn repair patterns [2, 17, 18]. APHP jointly analyzes patch code and commit descriptions to extract API post-handling specifications consisting of a target API, critical variable, post-operation, and path condition, and then applies corresponding path analyses to detect missing checks and resource-handling bugs [29]. APHP shows that commit descriptions can provide repair intent beyond the code change itself, although its specifications primarily describe checks or paired operations that should follow an API call.

SEAL compares the program-dependence graphs before and after a security fix and infers Linux interface specifications from changed interprocedural value-flow paths. Its specifications center on source-to-use reachability, path conditions, and use-site order, and are executed by a shared value-flow analysis [9]. TyPATCH differs from SEAL in both

the evidence used for rule construction and the resulting representation. SEAL derives value-flow relations from program-dependence changes. TyPATCH jointly uses the commit message, code diff, and source context to generate an explicit description of the tracked object, action, guard, state, and transition of a tpestate rule. State constraints expressed in a commit description or API contract, but not necessarily visible as a changed value-flow path, can therefore participate in rule construction.

We attempted to include SEAL in the quantitative evaluation, but encountered practical difficulties: its publicly available version depends on private commercial tools and is incompatible with the recent Linux kernel version used in our experiments. KNIGHTER reports the same limitations [9, 43].

6.2 LLM-Assisted Static Analysis

One line of work uses LLMs to provide semantic information to an existing static analyzer. LLift integrates LLM reasoning into a conventional analysis workflow for practical bug detection. IRIS uses an LLM to infer taint sources and sinks and integrates these specifications into whole-repository CodeQL analysis. LAMeD generates annotations for allocation and deallocation functions, which are then consumed by an existing memory-leak analyzer [23, 26, 37]. These approaches use LLM-supplied API models, annotations, or other semantic information within a predefined analysis task.

Other systems use patch-derived rules to locate candidates and then invoke an LLM to judge each candidate. SpecAuditor extracts audit specifications from historical patches to locate relevant code and guide model-based auditing. Bug-Stone summarizes a recurring error pattern from a repaired instance, uses program analysis to retrieve structurally similar candidates, and then asks an LLM whether each candidate shares the same root cause [28, 41]. These systems use explicit rules to reduce the LLM’s search space, but the scan still requires LLM reasoning over individual candidates.

KNIGHTER is the work most related to TyPATCH. It summarizes a bug pattern from a Linux repair patch, generates a checker implementation plan, and uses an LLM to implement a complete checker from CSA templates and helper functions. The generated code undergoes compilation repair and is evaluated on the buggy and fixed versions of the patch [43]. KNIGHTER demonstrates kernel-scale bug finding with LLM-generated, path-sensitive checkers.

Complete-checker generation offers strong expressiveness because callbacks, abstract states, object propagation, condition handling, and report logic can all be customized for an individual patch. At the same time, the model output contains both patch-specific defect semantics and general analyzer implementation. Templates, few-shot examples, and helper functions reduce framework-level burden, but each generated checker still composes its own callbacks, object representations, path states, and report logic.

TYPATCH changes what the model must produce. Instead of implementing a complete checker, the model describes which object to track and which program events change its state or constitute a violation. The shared backend binds this rule to the program and carries the object's state through aliases and control flow. All rules share analysis semantics, and generation errors remain confined to the rules.

7 Conclusion

This paper introduces TYPATCH, which generates structured tpestate rules from Linux kernel patches and executes them with a shared alias-aware, path-sensitive backend. Reusing rather than regenerating the analyzer makes construction more reliable and substantially cheaper. Across 100 fixes and three models, TYPATCH finds 559 distinct bugs in Linux v6.16, including 121 confirmed by kernel developers, while 236 of 249 rules preserve their seed's core defect relation.

On the matched 38 patches, TYPATCH uses 88.3–90.1% fewer generation tokens than KNIGHTER while producing more artifacts and more precise initial reports under both models. Structured rule generation therefore offers stable, low-cost, scalable analysis that grows with repair history.

Acknowledgments

Generative AI systems were used as experimental subjects in the reported rule-generation workflows and to assist language editing. The authors verified the paper's technical claims, code, and experimental results.

References

- [1] Glenn Ammons, Rastislav Bodík, and James R. Larus. 2002. Mining specifications. In *Proceedings of the 29th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, John Launchbury and John C. Mitchell (Eds.). ACM, 4–16. doi:10.1145/503272.503275
- [2] Johannes Bader, Andrew Scott, Michael Pradel, and Satish Chandra. 2019. Getafix: learning to fix bugs automatically. *Proc. ACM Program. Lang.* 3, OOPSLA (2019), 159:1–159:27. doi:10.1145/3360585
- [3] Jia-Ju Bai, Julia Lawall, Qiu-Liang Chen, and Shi-Min Hu. 2019. Effective Static Analysis of Concurrency Use-After-Free Bugs in Linux Device Drivers. In *Proceedings of the 2019 USENIX Annual Technical Conference (USENIX ATC)*, Dahlia Malkhi and Dan Tsafir (Eds.). USENIX Association, 255–268. <https://www.usenix.org/conference/atc19/presentation/bai>
- [4] Thomas Ball and Sriram K. Rajamani. 2001. Automatically Validating Temporal Safety Properties of Interfaces. In *Proceedings of the 8th International SPIN Workshop on Model Checking Software (SPIN) (Lecture Notes in Computer Science, Vol. 2057)*, Matthew B. Dwyer (Ed.). Springer, 103–122. doi:10.1007/3-540-45139-0_7
- [5] Kevin Bierhoff and Jonathan Aldrich. 2007. Modular tpestate checking of aliased objects. In *Proceedings of the 22nd Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, Richard P. Gabriel, David F. Bacon, Cristina Videira Lopes, and Guy L. Steele Jr. (Eds.). ACM, 301–320. doi:10.1145/1297027.1297050
- [6] Cristiano Calcagno, Dino Distefano, Jérémy Dubreil, Dominik Gabi, Pieter Hooimeijer, Martino Luca, Peter W. O'Hearn, Irene Papakonstantinou, Jim Purbrick, and Dulma Rodriguez. 2015. Moving Fast with Software Verification. In *Proceedings of the 7th International Symposium on NASA Formal Methods (NFM) (Lecture Notes in Computer Science, Vol. 9058)*, Klaus Havelund, Gerard J. Holzmann, and Rajeev Joshi (Eds.). Springer, 3–11. doi:10.1007/978-3-319-17524-9_1
- [7] Dan Carpenter and contributors. 2026. Smatch: A Static Analysis Tool for C. <https://github.com/error27/smatch>. Accessed August 2026.
- [8] Hao Chen and David A. Wagner. 2002. MOPS: an infrastructure for examining security properties of software. In *Proceedings of the 9th ACM Conference on Computer and Communications Security (CCS)*, Vijayalakshmi Atluri (Ed.). ACM, 235–244. doi:10.1145/586110.586142
- [9] Wei Chen, Bowen Zhang, Chengpeng Wang, Wensheng Tang, and Charles Zhang. 2025. Seal: Towards Diverse Specification Inference for Linux Interfaces from Security Patches. In *Proceedings of the 20th European Conference on Computer Systems (EuroSys)*. ACM, 1246–1262. doi:10.1145/3689031.3717487
- [10] Andy Chou, Junfeng Yang, Benjamin Chelf, Seth Hallem, and Dawson R. Engler. 2001. An Empirical Study of Operating System Errors. In *Proceedings of the 18th ACM Symposium on Operating Systems Principles (SOSP)*, Keith Marzullo and Mahadev Satyanarayanan (Eds.). ACM, 73–88. doi:10.1145/502034.502042
- [11] Manuvir Das, Sorin Lerner, and Mark Seigle. 2002. ESP: Path-Sensitive Program Verification in Polynomial Time. In *Proceedings of the 2002 ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, Jens Knoop and Laurie J. Hendren (Eds.). ACM, 57–68. doi:10.1145/512529.512538
- [12] Robert DeLine and Manuel Fähndrich. 2001. Enforcing High-Level Protocols in Low-Level Software. In *Proceedings of the 2001 ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, Michael Burke and Mary Lou Soffa (Eds.). ACM, 59–69. doi:10.1145/378795.378811
- [13] Dawson R. Engler, Benjamin Chelf, Andy Chou, and Seth Hallem. 2000. Checking System Rules Using System-Specific, Programmer-Written Compiler Extensions. In *Proceedings of the 4th Symposium on Operating Systems Design and Implementation (OSDI)*, Michael B. Jones and M. Frans Kaashoek (Eds.). USENIX Association, 1–16. <http://dl.acm.org/citation.cfm?id=1251230>
- [14] Dawson R. Engler, David Yu Chen, Seth Hallem, Andy Chou, and Benjamin Chelf. 2001. Bugs as Deviant Behavior: A General Approach to Inferring Errors in Systems Code. In *Proceedings of the 18th ACM Symposium on Operating Systems Principles (SOSP)*, Keith Marzullo and Mahadev Satyanarayanan (Eds.). ACM, 57–72. doi:10.1145/502034.502041
- [15] Stephen J. Fink, Eran Yahav, Nurit Dor, G. Ramalingam, and Emmanuel Geay. 2006. Effective tpestate verification in the presence of aliasing. In *Proceedings of the 2006 ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, Lori L. Pollock and Mauro Pezzè (Eds.). ACM, 133–144. doi:10.1145/1146238.1146254
- [16] David Gens, Simon Schmitt, Lucas Davi, and Ahmad-Reza Sadeghi. 2018. K-Miner: Uncovering Memory Corruption in Linux. In *Proceedings of the 25th Annual Network and Distributed System Security Symposium (NDSS)*. The Internet Society. doi:10.14722/ndss.2018.23326
- [17] Jiyong Jang, Abeer Agrawal, and David Brumley. 2012. ReDeBug: Finding Unpatched Code Clones in Entire OS Distributions. In *Proceedings of the 2012 IEEE Symposium on Security and Privacy (S&P)*. IEEE Computer Society, 48–62. doi:10.1109/SP.2012.13
- [18] Seulbae Kim, Seunghoon Woo, Heejo Lee, and Hakjoo Oh. 2017. VUDDY: A Scalable Approach for Vulnerable Code Clone Discovery. In *Proceedings of the 2017 IEEE Symposium on Security and Privacy (S&P)*. IEEE Computer Society, 595–614. doi:10.1109/SP.2017.62
- [19] Stefan Krüger, Sarah Nadi, Michael Reif, Karim Ali, Mira Mezini, Eric Bodden, Florian Göpfert, Felix Günther, Christian Weinert, Daniel Demmler, and Ram Kamath. 2017. CogniCrypt: supporting developers in using cryptography. In *Proceedings of the 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*,

- Grigore Rosu, Massimiliano Di Penta, and Tien N. Nguyen (Eds.). IEEE Computer Society, 931–936. doi:10.1109/ASE.2017.8115707
- [20] Stefan Krüger, Johannes Späth, Karim Ali, Eric Bodden, and Mira Mezini. 2018. CrySL: An Extensible Approach to Validating the Correct Usage of Cryptographic APIs. In *Proceedings of the 32nd European Conference on Object-Oriented Programming (ECOOP) (LIPICs, Vol. 109)*, Todd D. Millstein (Ed.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 10:1–10:27. doi:10.4230/LIPICs.ECOOP.2018.10
- [21] Julia Lawall and Gilles Muller. 2018. Coccinelle: 10 Years of Automated Evolution in the Linux Kernel. In *Proceedings of the 2018 USENIX Annual Technical Conference (USENIX ATC)*, Haryadi S. Gunawi and Benjamin C. Reed (Eds.). USENIX Association, 601–614. <https://www.usenix.org/conference/atc18/presentation/lawall>
- [22] Tien-Duy B. Le and David Lo. 2018. Deep specification mining. In *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, Frank Tip and Eric Bodden (Eds.). ACM, 106–117. doi:10.1145/3213846.3213876
- [23] Haonan Li, Yu Hao, Yizhuo Zhai, and Zhiyun Qian. 2024. Enhancing Static Analysis for Practical Bug Detection: An LLM-Integrated Approach. *Proc. ACM Program. Lang.* 8, OOPSLA1 (2024), 474–499. doi:10.1145/3649828
- [24] Tuo Li, Jia-Ju Bai, Yulei Sui, and Shi-Min Hu. 2022. Path-sensitive and alias-aware typestate analysis for detecting OS bugs. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Babak Falsafi, Michael Ferdman, Shan Lu, and Thomas F. Wenisch (Eds.). ACM, 859–872. doi:10.1145/3503222.3507770
- [25] Tuo Li, Jia-Ju Bai, Yulei Sui, and Shi-Min Hu. 2024. SPATA: Effective OS Bug Detection with Summary-Based, Alias-Aware, and Path-Sensitive Typestate Analysis. *ACM Trans. Comput. Syst.* 42, 3-4 (2024), 1–40. doi:10.1145/3695250
- [26] Ziyang Li, Saikat Dutta, and Mayur Naik. 2025. IRIS: LLM-Assisted Static Analysis for Detecting Security Vulnerabilities. In *Proceedings of the 13th International Conference on Learning Representations (ICLR)*. OpenReview.net. <https://openreview.net/forum?id=9LdJDU7E91>
- [27] Zhenmin Li and Yuanyuan Zhou. 2005. PR-Miner: automatically extracting implicit programming rules and detecting violations in large software code. In *Proceedings of the 10th European Software Engineering Conference held jointly with the 13th ACM SIGSOFT International Symposium on Foundations of Software Engineering (ESEC/FSE)*, Michel Wermelinger and Harald C. Gall (Eds.). ACM, 306–315. doi:10.1145/1081706.1081755
- [28] Miaoqian Lin and Hao Chen. 2026. SpecAuditor: Generating Audit Specifications for LLM-Driven Bug Detection. In *Proceedings of the 2026 IEEE Symposium on Security and Privacy (S&P)*. IEEE, 3396–3413. doi:10.1109/SP63933.2026.00206
- [29] Miaoqian Lin, Kai Chen, and Yang Xiao. 2023. Detecting API Post-Handling Bugs Using Code and Description in Patches. In *Proceedings of the 32nd USENIX Security Symposium (USENIX Security)*, Joseph A. Calandrino and Carmela Troncoso (Eds.). USENIX Association, 3709–3726. <https://www.usenix.org/conference/usenixsecurity23/presentation/lin>
- [30] V. Benjamin Livshits and Thomas Zimmermann. 2005. DynaMine: finding common error patterns by mining software revision histories. In *Proceedings of the 10th European Software Engineering Conference held jointly with the 13th ACM SIGSOFT International Symposium on Foundations of Software Engineering (ESEC/FSE)*, Michel Wermelinger and Harald C. Gall (Eds.). ACM, 296–305. doi:10.1145/1081706.1081754
- [31] LLVM Project. 2026. Clang Static Analyzer. <https://clang-analyzer.llvm.org/>. Accessed August 2026.
- [32] Kangjie Lu, Chengyu Song, Taesoo Kim, and Wenke Lee. 2016. UniSan: Proactive Kernel Memory Initialization to Eliminate Data Leakages. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi (Eds.). ACM, 920–932. doi:10.1145/2976749.2978366
- [33] Aravind Machiry, Chad Spensky, Jake Corina, Nick Stephens, Christopher Kruegel, and Giovanni Vigna. 2017. DR. CHECKER: A Soundy Analysis for Linux Kernel Drivers. In *Proceedings of the 26th USENIX Security Symposium (USENIX Security)*, Engin Kirda and Thomas Ristenpart (Eds.). USENIX Association, 1007–1024. <https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/machiry>
- [34] Tung Thanh Nguyen, Hoan Anh Nguyen, Nam H. Pham, Jafar M. Al-Kofahi, and Tien N. Nguyen. 2009. Graph-based mining of multiple object usage patterns. In *Proceedings of the 7th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT International Symposium on Foundations of Software Engineering (ESEC/FSE)*, Hans van Vliet and Valérie Issarny (Eds.). ACM, 383–392. doi:10.1145/1595696.1595767
- [35] Yoann Padioleau, Julia Lawall, René Rydhof Hansen, and Gilles Muller. 2008. Documenting and automating collateral evolutions in linux device drivers. In *Proceedings of the 3rd ACM SIGOPS/EuroSys European Conference on Computer Systems (EuroSys)*, Joseph S. Sventek and Steven Hand (Eds.). ACM, 247–260. doi:10.1145/1352592.1352618
- [36] Lucas Serrano, Van-Anh Nguyen, Ferdian Thung, Lingxiao Jiang, David Lo, Julia Lawall, and Gilles Muller. 2020. SPINFER: Inferring Semantic Patches for the Linux Kernel. In *Proceedings of the 2020 USENIX Annual Technical Conference (USENIX ATC)*, Ada Gavrilovska and Erez Zadok (Eds.). USENIX Association, 235–248. <https://www.usenix.org/conference/atc20/presentation/serrano>
- [37] Ekaterina Shemetova, Ivan Smirnov, Anton Alekseev, Ilya Shenbin, Alexey D. Rukhovich, Sergey I. Nikolenko, Vadim Lomshakov, and Irina Piontkovskaya. 2025. LAMeD: LLM-generated Annotations for Memory Leak Detection. In *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering (EASE)*, Muhammad Ali Babar, Ayse Tosun, Stefan Wagner, and Viktoria Stray (Eds.). ACM, 1024–1034. doi:10.1145/3756681.3756999
- [38] Robert E. Strom and Shaula Yemini. 1986. Typestate: A Programming Language Concept for Enhancing Software Reliability. *IEEE Trans. Software Eng.* SE-12, 1 (1986), 157–171. doi:10.1109/TSE.1986.6312929
- [39] Keita Suzuki, Kenta Ishiguro, and Kenji Kono. 2024. Balancing Analysis Time and Bug Detection: Daily Development-friendly Bug Detection in Linux. In *Proceedings of the 2024 USENIX Annual Technical Conference (USENIX ATC)*, Saurabh Bagchi and Yiyang Zhang (Eds.). USENIX Association, 493–508. <https://www.usenix.org/conference/atc24/presentation/suzuki>
- [40] Andrzej Wasylkowski, Andreas Zeller, and Christian Lindig. 2007. Detecting object usage anomalies. In *Proceedings of the 6th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT International Symposium on Foundations of Software Engineering (ESEC/FSE)*, Ivica Crnkovic and Antonia Bertolino (Eds.). ACM, 35–44. doi:10.1145/1287624.1287632
- [41] Qiushi Wu, Yue Xiao, Dhilung Kirat, Kevin Eykholt, Jiyong Jang, and Douglas Lee Schales. 2025. One Bug, Hundreds Behind: LLMs for Large-Scale Bug Discovery. *CoRR abs/2510.14036* (2025). arXiv:2510.14036 doi:10.48550/ARXIV.2510.14036
- [42] Yichen Xie and Alex Aiken. 2005. Scalable error detection using boolean satisfiability. In *Proceedings of the 32nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, Jens Palsberg and Martín Abadi (Eds.). ACM, 351–363. doi:10.1145/1040305.1040334
- [43] Chenyuan Yang, Zijie Zhao, Zichen Xie, Haoyu Li, and Lingming Zhang. 2025. KNighter: Transforming Static Analysis with LLM-Synthesized Checkers. In *Proceedings of the 31st ACM SIGOPS Symposium on Operating Systems Principles (SOSP)*, Youjip Won, Youngjin Kwon, Ding Yuan, and Rebecca Isaacs (Eds.). ACM, 655–669. doi:10.1145/3731569.3764827

- [44] Jinlin Yang, David Evans, Deepali Bhardwaj, Thirumalesh Bhat, and Manuvir Das. 2006. Perracotta: mining temporal API rules from imperfect traces. In *Proceedings of the 28th International Conference on Software Engineering (ICSE)*, Leon J. Osterweil, H. Dieter Rombach, and Mary Lou Soffa (Eds.). ACM, 282–291. doi:10.1145/1134285.1134325
- [45] Insu Yun, Changwoo Min, Xujie Si, Yeongjin Jang, Taesoo Kim, and Mayur Naik. 2016. APISan: Sanitizing API Usages through Semantic Cross-Checking. In *Proceedings of the 25th USENIX Security Symposium (USENIX Security)*, Thorsten Holz and Stefan Savage (Eds.). USENIX Association, 363–378. <https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/yun>

A Supplementary Details

A.1 Patch-to-Rule Algorithm

Algorithm A.1 summarizes evidence construction, structured generation, the two-stage validation pipeline, and field-level repair.

Algorithm A.1: Patch-to-Rule Construction

Input: Repair commit p , kernel source tree $\mathcal{K}$, rule schema $\mathcal{S}$

Output: Accepted typestate rule R_v , or NoRule

```

1 # Evidence construction
2  $c \leftarrow \text{BuildContext}(p.\text{message}, p.\text{diff}, \mathcal{K})$ 
3 # Structured rule generation
4  $y \leftarrow \text{GenerateRule}(c, \mathcal{S})$ 
5 # Two-stage validation and repair
6 for  $i \leftarrow 0$  to 2 do
7   if  $y = \text{NoRule}$  then
8     return NoRule
9    $F_g \leftarrow \emptyset; F_t \leftarrow \emptyset; F_b \leftarrow \emptyset$ 
10   $(R_c, F_s) \leftarrow \text{ValidateSchema}(y, \mathcal{S})$ 
11  if  $F_s = \emptyset$  then
12     $F_g \leftarrow \text{GroundBindings}(R_c, c, \mathcal{K})$ 
13  if  $F_s \cup F_g = \emptyset$  then
14     $R_v \leftarrow \text{NormalizeRule}(R_c, p.\text{diff})$ 
15     $F_t \leftarrow \text{ValidateStateMachine}(R_v)$ 
16  if  $F_s \cup F_g \cup F_t = \emptyset$  then
17     $F_b \leftarrow \text{ValidateBackend}(R_v)$ 
18    if  $F_b = \emptyset$  then
19       $\text{SerializeTS}(R_v)$ 
20      return  $R_v$ 
21  if  $i = 2$  then
22    return NoRule
23   $y \leftarrow \text{RepairRule}(c, y, F_s \cup F_g \cup F_t \cup F_b)$ 
24 return NoRule

```

A.2 Implementation Details

Patch-to-rule construction is implemented in Python, and the shared backend is implemented in C++ over LLVM IR.

The construction context includes at most three changed-function bodies, up to 200 lines each and 500 lines in total. Source expansion resolves ordinary functions, multi-line macros, one-level wrappers, and kernel `DEFINE_FREE` cleanup declarations, and follows one level of callees from those definitions. Every excerpt retains `file:line` provenance and is available to the grounding checks after generation. For few-shot prompting, we select at most two patch-to-rule examples by family features from a frozen set whose rules have already executed on the backend; no example is supplied when no family match exists.

The model-facing schema contains R . Its state domain records declared join cases, while the Φ part of K records report constraints. A deterministic serializer completes J_Q , maps Φ to the backend’s fail-open path-screening policy, emits the `.ts` format, and attaches the selected scan settings. When a patch names an allocation or release function, context construction may also provide related kernel functions for the model to consider. The generated IR records the functions selected by the model, and the normalized `.ts` rule records the executable bindings.

A.3 Evaluation Protocol

Rule scope and scan configuration. Some patch-derived rules depend on module-specific API or lifecycle conventions. We therefore assign each rule either kernel scope or module scope before scanning. Kernel scope covers all link targets in the compilation database, whereas module scope selects a subset of those targets. The initial scope is derived from the source distribution of function bindings in the generated and executable rules. Generic or unresolved bindings lead to kernel scope. An LLM then retains or widens an initially local scope, but cannot narrow it. Rules with identical target sets are executed together to reduce unnecessary rule–target combinations. Both RQ1 and RQ2 enable this step for Ty-PATCH.

Report collection and review. We used a two-hour limit per translation unit. RQ1 merges reports by source site and its connected downstream sinks. Multiple sinks reached from one source form one finding; components from different rules or object keys are joined transitively when they share a sink. Reports at the original seed sites and in kernel test code are excluded from production-bug totals. To keep exhaustive source review feasible, the RQ1 report pools exclude reports from any rule whose whole-kernel scan produces more than 2,000 raw reports. This removes 2 of 91 GPT rules, 1 of 74 DeepSeek rules, and 3 of 84 Opus rules. RQ2 applies no report-count gate.

The matched set contains the 38 commits in KNIGHTER’s public collection that belong to the six sequential object-state families represented by both systems: 6 null-dereference, 5 leak, 7 use-after-release, 8 double-free, 5 uninitialized-data, and 7 misuse fixes. RQ2 deduplicates separately within

each rule or checker by sink file, function, and line; reports from different generated artifacts remain distinct. We use KNIGHTER commit f4e834b30741 and its published default configuration. TyPATCH follows the construction procedure in Section 3.2; all generation tokens from both workflows are counted. Report refinement and triage are outside both artifact-generation arms.

All reported populations were manually reviewed against the same Linux v6.16 source. A report counts as a true positive only when its ordered actions operate on the same abstract object along a feasible path.

Model-usage accounting. For TyPATCH, the generation-token totals reported in RQ1 and RQ2 include initial rule generation, all repair attempts, and LLM-based scope planning; the reported model-call counts likewise include scope-planning calls. Token totals use provider-reported input, cached-input, and output counts. Backend execution uses the saved scope maps without further model calls, and manual review does not consume model tokens.

A.4 Additional Results

Table 5. RQ1 rule-construction cost, in thousands of model tokens. Totals include initial generation, repair attempts, and scope planning.

Model	Rules	Input	Output	Total
GPT-5.5	91	2,229	294	2,523
DeepSeek-v4-pro	74	1,955	956	2,911
Opus 4.8	84	2,964	152	3,116

Table 6. RQ2 true-positive report instances by family.

Family	GPT-5.5		DeepSeek-v4-pro	
	TyPATCH	KNIGHTER	TyPATCH	KNIGHTER
Memory/Resource Leak	138	1	141	7
Use After Release	30	2	0	6
Uninitialized Data	32	4	0	20
Null-Pointer Dereference	14	7	17	12
Double Free	0	1	0	3
Misuse	0	0	0	10
Total	214	15	158	58

Table 6 gives the family-level composition behind the aggregate RQ2 results. Because reports from different artifacts remain distinct, these entries are report instances rather than distinct bugs.

Within TyPATCH, DeepSeek-v4-pro achieves higher precision than GPT-5.5 (9.11% vs. 4.67%). The difference is dominated by three overly broad GPT-5.5 rules, which produce 2,729 of the 2,849 additional reports relative to DeepSeek-v4-pro. One rule approximates a concurrent use-after-release as a sequential free-use relation and produces 1,753 reports; two uninitialized-data rules produce 545 and 431. Their false positives arise primarily from different-object propagation, infeasible paths, and guards or initialization that precede the reported use.